

**SIMATS ENGINEERING  
ASSESSMENT TOOL 2**

**COURSE NAME: Design and Analysis of Algorithms**  
**COURSE CODE: CSA06 12**

---

**COURSE OUTCOME COVERED**

**CO2:** Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1,BL2,BL3)

**Assessment Tool Weightage: 20%**

---

**QUESTIONS: 10**

**Total Marks: 50**

Annexure A

**Scenario Based Assessment**

---

***Code of Conduct:***

*I Yaswanthika Shree S (192565069) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

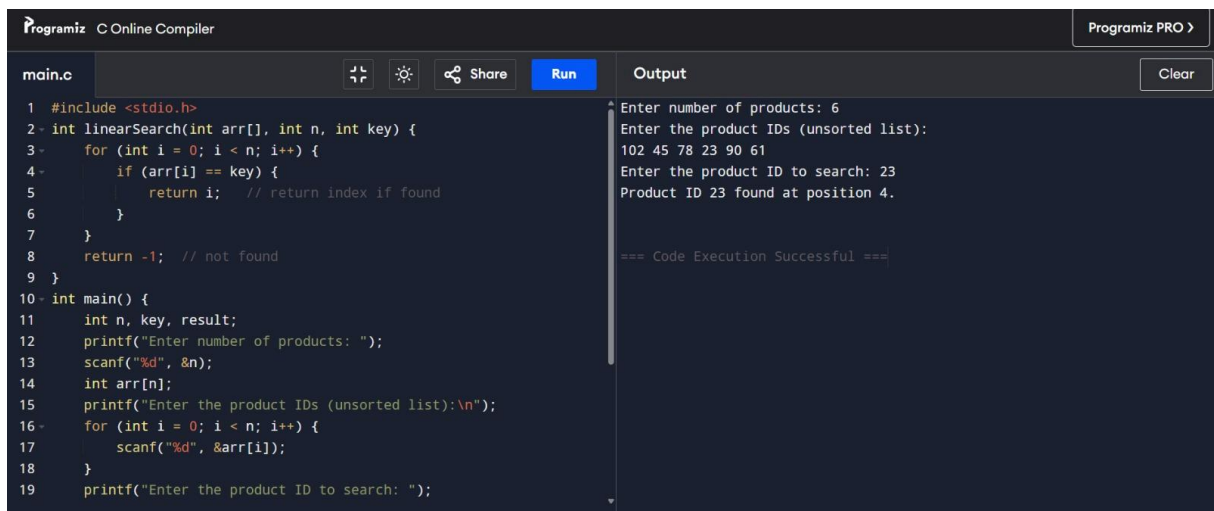
***Signature of the student: Yaswanthika Shree S***

## Annexure A

### Scenario-Based Assessment

**Q36. A retail chain maintains a product list that is frequently updated and unsorted. The system searches for products using Linear Search.**

- Explain how Linear Search operates in this system.
- Analyze its performance as dataset size increases.
- Suggest whether Binary Search would be beneficial and justify.

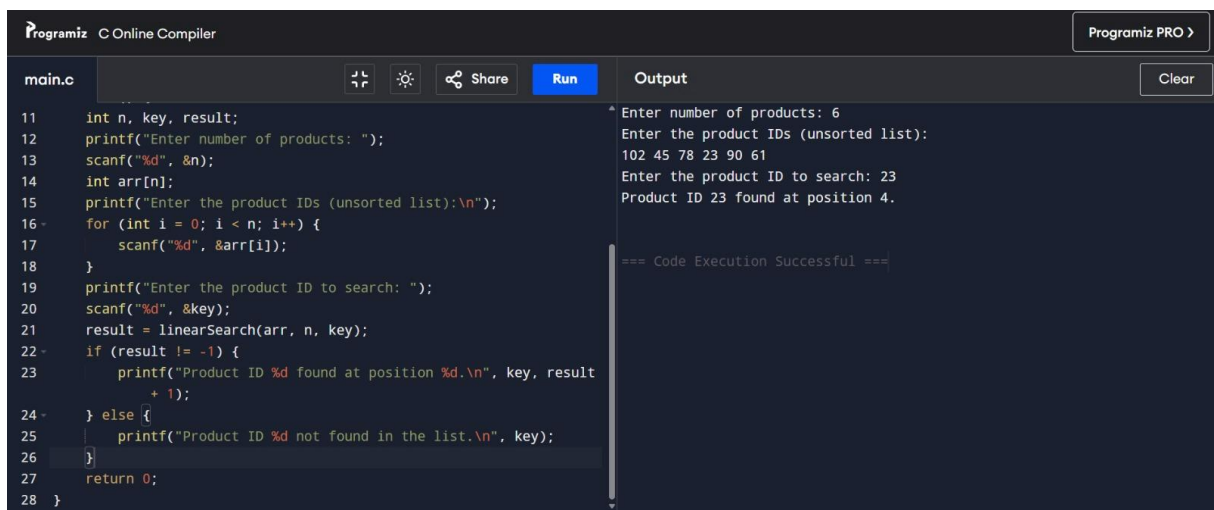


```
1 #include <stdio.h>
2 int linearSearch(int arr[], int n, int key) {
3     for (int i = 0; i < n; i++) {
4         if (arr[i] == key) {
5             return i; // return index if found
6         }
7     }
8     return -1; // not found
9 }
10 int main() {
11     int n, key, result;
12     printf("Enter number of products: ");
13     scanf("%d", &n);
14     int arr[n];
15     printf("Enter the product IDs (unsorted list):\n");
16     for (int i = 0; i < n; i++) {
17         scanf("%d", &arr[i]);
18     }
19     printf("Enter the product ID to search: ");
```

Output

```
Enter number of products: 6
Enter the product IDs (unsorted list):
102 45 78 23 90 61
Enter the product ID to search: 23
Product ID 23 found at position 4.

=== Code Execution Successful ===
```



```
11 int n, key, result;
12 printf("Enter number of products: ");
13 scanf("%d", &n);
14 int arr[n];
15 printf("Enter the product IDs (unsorted list):\n");
16 for (int i = 0; i < n; i++) {
17     scanf("%d", &arr[i]);
18 }
19 printf("Enter the product ID to search: ");
20 scanf("%d", &key);
21 result = linearSearch(arr, n, key);
22 if (result != -1) {
23     printf("Product ID %d found at position %d.\n", key, result + 1);
24 } else {
25     printf("Product ID %d not found in the list.\n", key);
26 }
27 return 0;
28 }
```

Output

```
Enter number of products: 6
Enter the product IDs (unsorted list):
102 45 78 23 90 61
Enter the product ID to search: 23
Product ID 23 found at position 4.

=== Code Execution Successful ===
```

(a) The linearSearch function checks each element one by one from index 0 until it finds the key or reaches the end — that's exactly how the retail system would scan its unsorted, frequently-updated product list.

(b) You can demonstrate performance by increasing  $n$  (e.g.,  $10 \rightarrow 1000 \rightarrow 100000$  products) and observing that search time grows linearly ( $O(n)$ ) — worst case when the product is last or absent.

(c) Since the list is frequently updated (insertions/deletions), keeping it sorted for Binary Search would add extra overhead (re-sorting on every update), so Linear Search is often justified despite being  $O(n)$ , unless updates are infrequent relative to searches — in which case Binary Search ( $O(\log n)$ ) becomes worthwhile.

## RUBRICS FOR CALCULATION

| Criteria                                 | Weightage | Level 4 – Exemplary                                                                  | Level 3 – Proficient                                  | Level 2 – Developing                                      | Level 1 – Beginning                                     |
|------------------------------------------|-----------|--------------------------------------------------------------------------------------|-------------------------------------------------------|-----------------------------------------------------------|---------------------------------------------------------|
| Problem Understanding & Context Analysis | 25        | Thoroughly understands the scenario with accurate interpretation of all requirements | Clearly understands the scenario with minor gaps      | Basic understanding; some aspects of the scenario unclear | Poor understanding or misinterpretation of the scenario |
| Application of Concepts                  | 30        | Applies appropriate concepts effectively to solve complex real-world problems        | Applies relevant concepts correctly with minor errors | Applies basic concepts but with limited accuracy          | Fails to apply appropriate concepts                     |
| Analytical & Decision-Making Skills      | 20        | Demonstrates strong analysis and selects optimal solutions with justification        | Good analysis with reasonable solutions               | Limited analysis; solutions lack justification            | Poor or no analysis; incorrect decisions                |
| Solution Design & Approach               | 15        | Well-structured, innovative, and feasible solution approach                          | Logical and structured solution with minor gaps       | Basic solution with limited structure or feasibility      | Unclear or incorrect solution approach                  |
| Clarity, Justification & Presentation    | 10        | Clear, well-justified explanation with strong reasoning                              | Clear explanation with some justification             | Limited clarity and weak justification                    | Poorly explained with no justification                  |